

EE2026

Digital Design

NUMBER SYSTEMS

Massimo ALIOTO

Dept of Electrical and Computer Engineering

Email: massimo.alioto@nus.edu.sg

Get to know the latest silicon system breakthroughs from our labs in 1-minute video demos



Outline

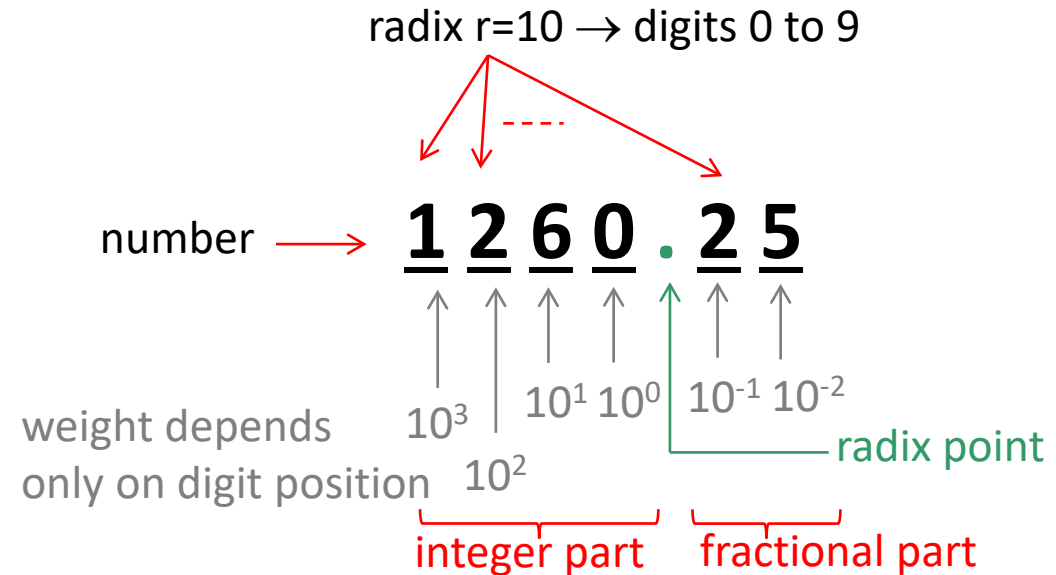
- Positional number systems
- Radix conversion
- Binary arithmetic
- Binary signed representation
- Binary-coded decimal (BCD)

Positional Number Systems

- Decimal number system: radix $r=10$

Terminology

- Radix (or base)
- Radix point
- Numerals made of digits ($0 \rightarrow r-1$)
- Place value (or weight) is in the power of the base (positive on the left and negative on the right side of the radix point)

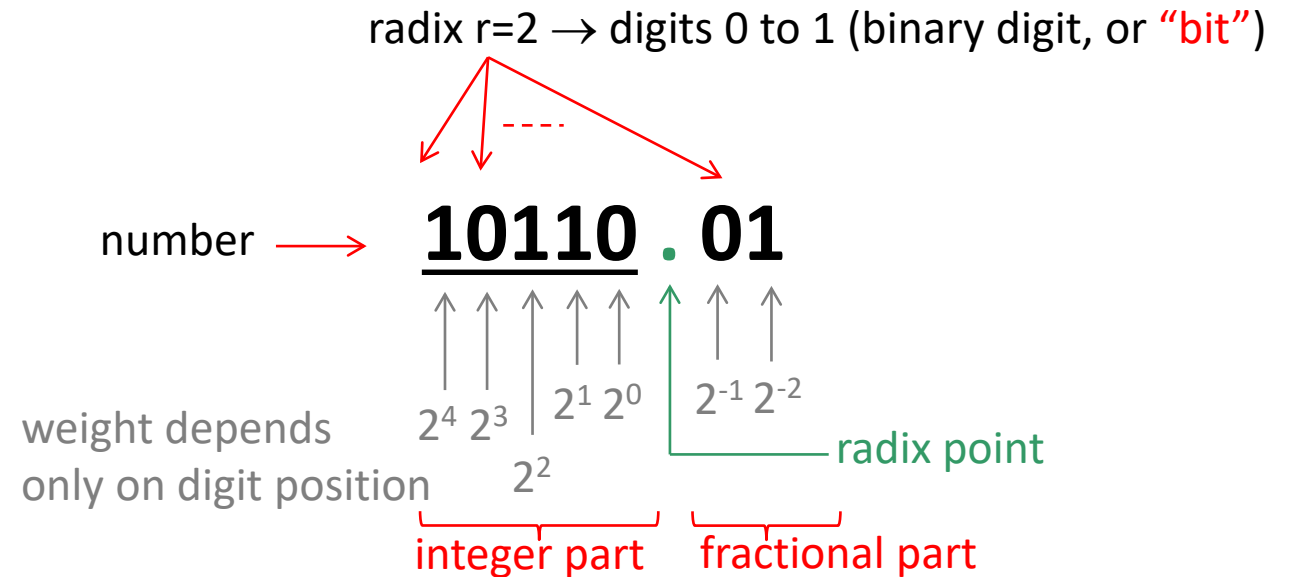


$$N = 1 \times 10^3 + 2 \times 10^2 + 6 \times 10^1 + 0 \times 10^0 + 2 \times 10^{-1} + 5 \times 10^{-2} = 1260.25$$

weighted sum of each digit (each digit is weighted by its place value)

Binary Number System

- Binary number system: radix $r=2$

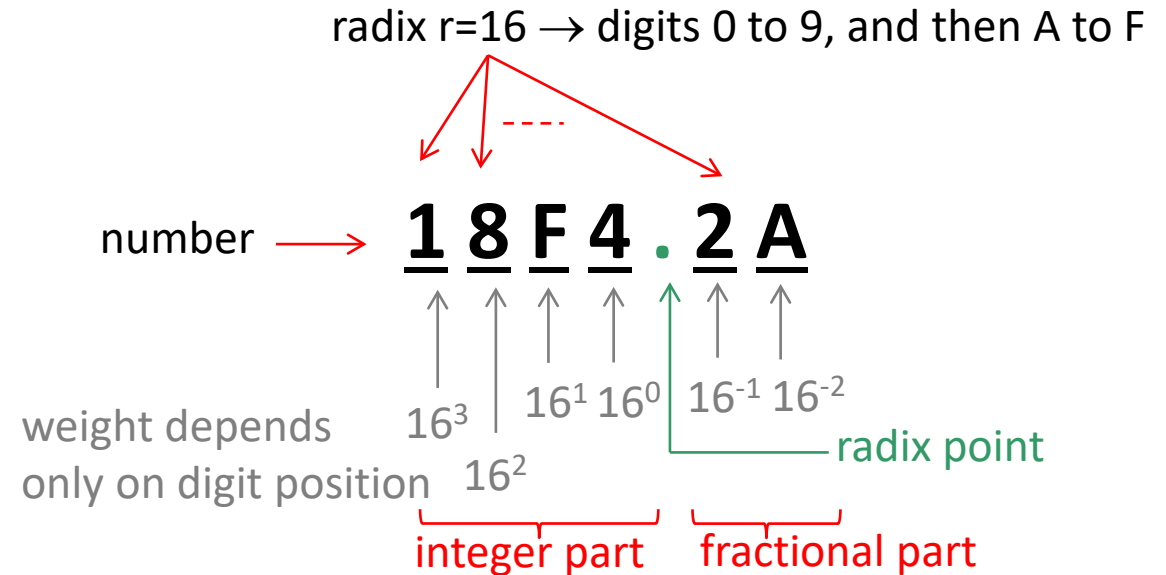


- Decimal equivalent

$$\begin{aligned}(N)_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ &= 16 + 0 + 4 + 2 + 0 + 0 + \frac{1}{4} \\ (N)_{10} &= 22.25\end{aligned}$$

Hexadecimal Number System

- Hexadecimal number system: radix $r=16$



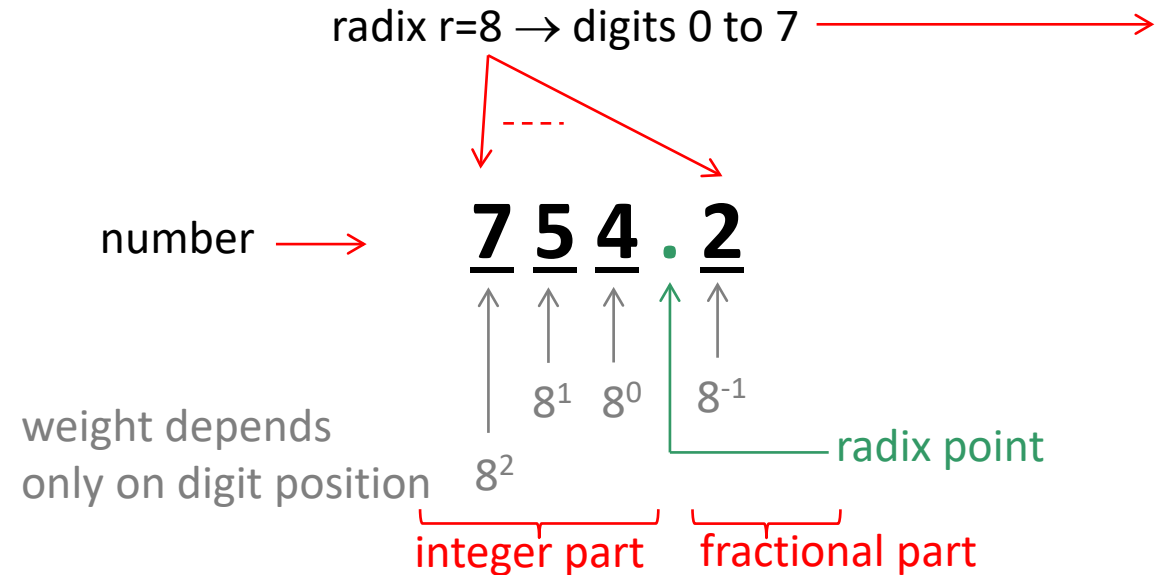
- Decimal equivalent

$$\begin{aligned}
 (N)_{16} &= 1 \times 16^3 + 8 \times 16^2 + F \times 16^1 + 4 \times 16^0 + 2 \times 16^{-1} + 10 \times 16^{-2} \\
 &= 4096 + 2048 + 240 + 4 + \frac{2}{16} + \frac{10}{256} \\
 (N)_{10} &= 6388 + \frac{21}{128} \approx 6388.16
 \end{aligned}$$

hex	dec
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Octal Number System

- Octal number system: radix $r=8$



Oct	Dec
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
?	8
?	9
?	10

- Decimal equivalent

$$(N)_8 = 7 \times 8^2 + 5 \times 8^1 + 4 \times 8^0 + 2 \times 8^{-1}$$

$$(N)_{10} = 448 + 40 + 4 + \frac{2}{8} \approx 492.25$$

Generalization to Any Positional Number System

- Generic number system with radix r

radix $r \rightarrow$ digits 0 to $r-1$

$$(A)_r = (a_n a_{n-1} \dots a_0 . a_{-1} \dots a_{-m})_r$$

... r^n r^{n-1} ... r^0 r^{-1} ... r^{-m} radix point

- Decimal equivalent is weighted sum of all digits

$$(A)_r = (a_n a_{n-1} \dots a_0 . a_{-1} \dots a_{-m})_r$$

$$(A)_{10} = a_n \times r^n + a_{n-1} \times r^{n-1} + \dots + a_0 \times r^0 + a_{-1} \times r^{-1} + \dots + a_{-m} \times r^{-m}$$

$$= \sum_{i=-m}^n a_i r^i$$

Radix Conversion

- Three types of conversions

- Radix r ($r \neq 10$) $\rightarrow$ decimal
- Decimal $\rightarrow$ radix r ($r \neq 10$)
- Conversion among binary, octal and hex numbers

- Radix r ($r \neq 10$) $\rightarrow$ decimal is trivial (decimal equivalent) $(A)_{10} = \sum_{i=-m}^n a_i r^i$
 - Binary $\rightarrow$ decimal

$$\begin{aligned}(10110.01)_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ &= 16 + 0 + 4 + 2 + 0 + 0 + \frac{1}{4} = (22.25)_{10}\end{aligned}$$

- Hex $\rightarrow$ decimal

$$\begin{aligned}(18F4.2A)_{16} &= 1 \times 16^3 + 8 \times 16^2 + F \times 16^1 + 4 \times 16^0 + 2 \times 16^{-1} + 10 \times 16^{-2} \\ &= 6388 + \frac{21}{128} \approx (6388.16)_{10}\end{aligned}$$

Decimal → Radix r (r≠10): Integer Part

- Must find a systematic way

- Decimal → binary $(102)_{10} = (A)_2 = (a_n a_{n-1} \dots a_0 . a_{-1} \dots a_{-m})_r = ?$

- Start simple: integer number

$$(102)_{10} = (A)_2 = a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_1 \times 2^1 + a_0$$

$$= \underbrace{(a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_1 \times 2^1)}_{\text{integer multiple of 2}} + a_0$$

integer multiple of 2

integer multiple of 2 if $a_0=0$,
(integer multiple of 2)+1 if $a_0=1$

→ a_0 is remainder of division by 2

→ quotient $a_n \times 2^{n-1} + a_{n-1} \times 2^{n-2} + \dots + a_1$ continue dividing quotient by 2

$$2 \overline{) \begin{array}{l} a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_1 \times 2 + a_0 \\ \underline{a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_1 \times 2} \end{array}}$$

remainder is a_0

$$2 \overline{) \begin{array}{l} a_n \times 2^{n-2} + a_{n-1} \times 2^{n-3} + \dots + a_1 \\ \underline{a_n \times 2^{n-1} + a_{n-1} \times 2^{n-2} + \dots + a_1} \\ \underline{a_n \times 2^{n-1} + a_{n-1} \times 2^{n-2} + \dots} \end{array}}$$

remainder is a_1

- Numerical example of repeated division by 2: $(102)_{10} = (A)_2 = (?)_2$

division by 2	quotient	remainder
102/2	51	$0 \rightarrow a_0$
51/2	25	$1 \rightarrow a_1$
25/2	12	$1 \rightarrow a_2$
12/2	6	$0 \rightarrow a_3$
6/2	3	$0 \rightarrow a_4$
3/2	1	$1 \rightarrow a_5$
1/2	0	$1 \rightarrow a_6$

stop divisions when quotient=0
(subsequent divisions are all zero)

$$(A)_2 = \dots 00001100110$$

Check (inverse calculation)

$$\begin{aligned}
 (A)_{10} &= a_6 \times 2^6 + a_5 \times 2^5 + a_4 \times 2^4 + a_3 \times 2^3 \\
 &\quad + a_2 \times 2^2 + a_1 \times 2^1 + a_0 \times 2^0 \\
 &= 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 \\
 &\quad + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\
 &= 64 + 32 + 0 + 0 + 4 + 2 + 0 \\
 &= 102 \checkmark
 \end{aligned}$$

Decimal → Radix r (r≠10): Fractional Part

- Must find a systematic way for the fractional part too
 - Then, sum up with integer part for arbitrary numbers with integer and fractional part

- Fractional number

$$(0.58)_{10} = (A)_2 = (0.a_{-1}a_{-2} \dots a_{-m+1}a_{-m})_2 = \\ = a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \dots + a_{-m+1} \times 2^{-m+1} + a_{-m} \times 2^{-m}$$

- Interesting interpretation of a_{-1} after multiplying the number by 2

$$2 \times (a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \dots + a_{-m+1} \times 2^{-m+1} + a_{-m} \times 2^{-m}) = \\ = \underbrace{a_{-1}}_{\text{integer part of } 2 \times (A)_{10}} + a_{-2} \times 2^{-1} + \dots + a_{-m+1} \times 2^{-m+2} + a_{-m} \times 2^{-m+1}$$

integer part of $2 \times (A)_{10}$

fractional part of $2 \times (A)_{10}$

a_{-1} can be simply found as the integer part of $2 \times (A)_{10}$,
then repeat for a_{-2} , a_{-3} ...

- Numerical example of repeated multiplications by 2: $(0.58)_{10} = (A)_2 = (?)_2$

multiply by 2	product	integer part
0.58x2	1.16	1 $\rightarrow a_{-1}$
0.16x2	0.32	0 $\rightarrow a_{-2}$
0.32x2	0.64	0 $\rightarrow a_{-3}$
0.64x2	1.28	1 $\rightarrow a_{-4}$
0.28x2	0.56	0 $\rightarrow a_{-5}$
0.56x2	1.12	1 $\rightarrow a_{-6}$
0.12x2	0.24	0 $\rightarrow a_{-7}$
0.24x2	0.48	0 $\rightarrow a_{-8}$

Check (inverse calculation)

$$\begin{aligned}
 (A)_{10} &= 1 \times 2^{-1} + 1 \times 2^{-4} + 1 \times 2^{-6} \\
 &= \frac{1}{2} + \frac{1}{16} + \frac{1}{64} \\
 &= 0.578125 \\
 &\approx 0.58 \checkmark
 \end{aligned}$$

- $(A)_2 = 10010100... \leftarrow$
- conversion certainly stops when product=0
 - may never end (non-terminating representation, even if $(A)_{10}$ may terminate)
 - in this case, stop at the required precision (system specification, no. of bits)

Summary Table for Conversion of 4-bit Integers

Numbers with Different Radixes

Decimal (radix 10)	Binary (radix 2)	Octal (radix 8)	Hexadecimal (radix 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Conversion among Hex, Octal and Binary

- Hex $\leftrightarrow$ binary
 - Each Hex digit $\rightarrow$ 4 bits
 - Or each group of 4 bits $\rightarrow$ 1 Hex digit (starting from radix point and add zeroes if necessary to have all groups of 4 bits)
- Octal $\leftrightarrow$ binary
 - Each octal digit (0...7) $\rightarrow$ 3 bits (indeed 0...7)
 - Or each group of 3 bits $\rightarrow$ 1 octal digit (starting from radix point)
- Hex $\leftrightarrow$ Octal
 - Use binary as an intermediate step
 - Hex $\rightarrow$ binary $\rightarrow$ octal
 - Octal $\rightarrow$ binary $\rightarrow$ Hex

Numbers with Different Radixes

Decimal (radix 10)	Binary (radix 2)	Octal (radix 8)	Hexadecimal (radix 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Examples (Hex, Octal, Binary Conversion)

Hex → bin:

$(A45F)_{16}$

$(\underline{1010} \ \underline{0100} \ \underline{0101} \ \underline{1111})_2$

Oct → bin:

$(475)_8$

$(\underline{100} \ \underline{111} \ \underline{101})_2$

Bin → Hex:

$00(\underline{11} \ \underline{1010} \ \underline{1101} \ \underline{0111})_2$

complete with zeroes on
the left in integer numbers
(same number), on the
right in fractional numbers

$(3 \ A \ D \ 7)_{16}$

Bin → oct:

$0(\underline{10} \ \underline{111} \ \underline{101} \ \underline{110})_2$

$(2 \ 7 \ 5 \ 6)_8$

Other Examples (Hex, Octal, Binary Conversion)

Hex → oct:

$(A45F)_{16}$

$(\underline{1010} \ \underline{0100} \ \underline{0101} \ \underline{1111})_2$

00 $(\underline{1} \ \underline{010} \ \underline{010} \ \underline{001} \ \underline{011} \ \underline{111})_2$

regroup in 3-bit groups, add zeroes

↓
convert each 3-bit group

$(122137)_8$

Oct → Hex:

$(653)_8$

$(\underline{110} \ \underline{101} \ \underline{011})_2$

000 $(\underline{1} \ \underline{1010} \ \underline{1011})_2$

↓

$(1AB)_{16}$

- For the fractional part: same, just group digits by starting from the position after the radix point (same number when adding zeroes on the right)

Hex → bin:

$(0.A45F)_{16}$

$(0.\underline{1010} \ \underline{0100} \ \underline{0101} \ \underline{1111})_2$ **00...**

add zeroes to the right
(same number, not
needed in this example)

Radix Conversion: Summary and Generalization

- Radix r ($r \neq 10$) $\rightarrow$ decimal
 - Just use definition of positional number system (weighted sum of all digits)
- Decimal $\rightarrow$ Radix r ($r \neq 10$)
 - Integer part $\rightarrow$ repeated division by r and take the remainder
 - Fractional part $\rightarrow$ repeated multiplication by r and take the integer part
 - Add integer and fractional part
- Conversion among binary, octal and Hex numbers
 - Grouping: 1 Hex digit = 4 bits, 1 octal digit = 3 bits
 - Conversion with binary as intermediate step + regroup: Hex $\rightarrow$ oct carried out as Hex $\rightarrow$ Binary $\rightarrow$ Octal, and vice versa

Binary Arithmetic

- Addition
- Multiplication
- Subtraction
- Division
- Arithmetic operations in digital systems

Addition

- Start simple: 1-bit addition

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 1 = 10$$



“1” is the carry to the next higher bit

- Multi-bit addition: same rules as addition in decimal system

$$10111 + 110 = 11101$$

$$\begin{array}{r} \leftarrow \text{carry} \\ 10111 \\ + 110 \\ \hline 11101 \end{array}$$

Multiplication

- 1-bit multiplication

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 1 = 1$$

- Multi-bit multiplication: same rules as multiplication in decimal system

- Interesting: multiplication is basically shift and add (partial products)
 - shift is for free (just wiring)
 - multiplier can be simplified into an inexpensive adder

$$10111 \times 110 = 10001010$$

1 0 1 1 1	← multiplicand
x 1 1 0	← multiplier

0 0 0 0 0	} ← partial products
1 0 1 1 1	
+ 1 0 1 1 1	

1 0 0 0 1 0 1 0	← product

Subtraction

- 1-bit subtraction

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

$$0 - 1 = 1 \leftarrow \text{with a borrow from the next (higher) bit position}$$

- Multi-bit subtraction: same rules as subtraction in decimal system

$$11011 - 110 = 10101$$

$$\begin{array}{r} \text{-1} \leftarrow \text{borrow} \\ 1\ 1\ 0\ 1\ 1 \\ -\quad 1\ 1\ 0 \\ \hline 1\ 0\ 1\ 0\ 1 \end{array}$$

Division

- Division: same rules as division in decimal system

$$100101/101 = ?$$

$$\begin{array}{r} 101 \overline{) 100101} \\ \underline{101} \\ 1000 \\ \underline{101} \\ 111 \\ \underline{101} \\ 10 \end{array}$$

- **Set** quotient to 0
- Align leftmost digits in dividend and divisor
- **Repeat**
 - **If** that portion of the dividend above the divisor is greater than or equal to the divisor
 - **Then** subtract divisor from that portion of the dividend and
 - Concatenate 1 to the right hand end of the quotient
 - **Else** concatenate 0 to the right hand end of the quotient
 - Shift the divisor one place right
- **Until** dividend is less than the divisor
- quotient is correct, dividend is remainder
- **STOP**

- Interesting: multiplication is basically shift and subtract
 - shift is for free (just wiring) → divider can be simplified into an inexpensive subtractor

Arithmetic in Digital Systems and Simplifications

- Only addition and subtraction are needed for all four binary arithmetic operations
- Subtraction can be performed by adding a negative number
- A digital system potentially needs only adders to perform all binary arithmetic operations
- CHALLENGE: this requires an appropriate representation of the negative binary numbers to inexpensively carry out subtractions
 - reuse unsigned adder for signed operations

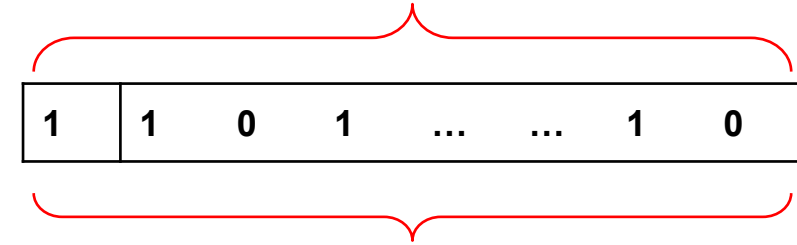
Unsigned Binary Numbers

- n -bit unsigned number
 - always non-negative
 - no sign (all bits contribute to magnitude)
- Range of n -bit unsigned binary number
 - Lowest (00...0): **0**
 - Highest (11...1): $(2^n - 1)_{10}$
 - 2^n values starting from 0 $\rightarrow$ **0... $2^n - 1$**

4-bit example:

$$\overset{2^4}{1} \overset{2^3}{0} \overset{2^2}{0} \overset{2^1}{0} \overset{2^0}{0} - 1 \rightarrow (2^4)_{10} - 1$$

unsigned binary number (n bits)



magnitude

Decimal	Unsigned binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Signed Binary Numbers

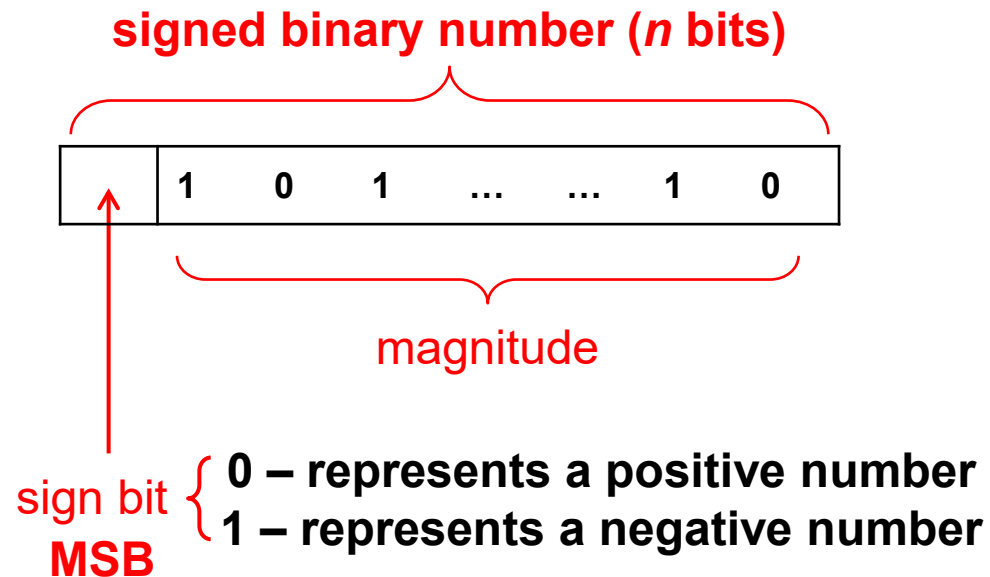
- Three representations of signed binary numbers
 - Sign + magnitude
 - 1's complement
 - 2's complement
- Naming of bits based on their position
 - Most significant bit (MSB): position with highest weight → left-most
 - Least significant bit (LSB): position with lowest weight → right-most

1 0 1 1 0 0 1

(left-most bit) MSB ↑ ↑ LSB (right-most bit)

Signed Binary Numbers: Signed-Magnitude (SM)

- MSB is the sign
- Subsequent bits are the magnitude



- Two different representations of zero
 - Very inconvenient: comparison with zero is used very frequently (1 comparison → 2 comparisons)

3-bit example

Decimal	S-M
3	011
2	010
1	001
+0	000
-0	100
-1	101
-2	110
-3	111

note:
two zeros

negative
numbers

“1” in MSB position for all negative numbers

Signed Binary Numbers: Signed-Magnitude (SM)

- Some more examples:
$$00111010 = +0111010 = (58)_{10}$$
$$11100101 = -1100101 = (-101)_{10}$$
$$10000001 = -0000001 = (-1)_{10}$$
$$01111111 = +1111111 = (+127)_{10}$$
- Range of n -bit sign-magnitude binary number
 - Lowest (11....1)
 - magnitude is $(n-1)$ -bit unsigned number $(0 \dots 2^{n-1})$
 - lowest = $-(\text{max magnitude}) = -(2^{n-1}-1)_{10}$
 - Highest (01....1): just the opposite
 - highest = $+(2^{n-1}-1)_{10}$
 - Range: $-(2^{n-1}-1)_{10} \rightarrow (2^{n-1}-1)_{10}$

Decimal	S-M
3	011
2	010
1	001
+0	000
-0	100
-1	101
-2	110
-3	111

Arithmetic with Sign-Magnitude Binary Numbers

- Subtraction equivalent to adding a negative number
- In sign-magnitude representation, can we reuse simple unsigned adder for subtraction?
 - Provably not possible, if we find just one case where this is not true
 - Some (counter)examples

$$\begin{array}{r} 3 \quad 011 \\ - 2 \quad 110 \\ \hline 1 \quad 1001 \end{array}$$

discarded

(1)₁₀ ✓

$$\begin{array}{r} 3 \quad 011 \\ - 1 \quad 101 \\ \hline 2 \quad 1000 \end{array}$$

discarded

(0)₁₀ ✗

$$\begin{array}{r} 2 \quad 010 \\ - 1 \quad 101 \\ \hline 1 \quad 111 \end{array}$$

(-3)₁₀ ✗

- Sign-magnitude representation cannot reuse unsigned adder for two operands with opposite signs
 - dedicated subtractor would be needed, two zero representations
→ need better representation

Diminished-Radix Complement Representation

- A “new” operation: diminished radix complement A^* of an n -digit integer A with radix r defined as

$$A^* = r^n - 1 - A$$

- Example: diminished radix complement of binary integer A
→ 1's complement $A^* = 2^n - 1 - A$

Decimal

$$A = 2375 \rightarrow A^* = (10000_{10} - 1) - 2375_{10} = 9999_{10} - 2375_{10} = 7624_{10}$$

$$A = 0919 \rightarrow A^* = (10000_{10} - 1) - 0919_{10} = 9080_{10}$$

Octal

$$A = 406 \rightarrow A^* = (1000_8 - 1) - 406_8 = 777_8 - 406_8 = 371_8$$

$$A = 0671 \rightarrow A^* = (10000_8 - 1) - 0671_8 = 7777_8 - 0671_8 = 7106_8$$

Hex

$$A = 4A09 \rightarrow A^* = (10000_{16} - 1) - 4A09_{16} = FFFF_{16} - 4A09_{16} = B5F6_{16}$$

$$A = 0A7F \rightarrow A^* = (10000_{16} - 1) - 0A7F_{16} = FFFF_{16} - 0A7F_{16} = F580_{16}$$

Binary (1's complement)

$$A = 1001 \rightarrow A^* = (10000_2 - 1) - 1001_2 = 1111_2 - 1001_2 = 0110_2$$

$$A = 1100 \rightarrow A^* = (10000_2 - 1) - 1100_2 = 1111_2 - 1100_2 = 0011_2$$

← **2' complement simply
is found by reversing all bits**

Diminished-Radix Complement Representation

- 1's complement representation of n -bit signed (integer) binary numbers
 - Number defined by sign and magnitude $\rightarrow$ start from magnitude A
 - if $A \geq 0$, just take the magnitude (n bits)
 - if $A < 0$, take the 1's complement of the magnitude (n bits)
- Example: can we reuse simple unsigned adder for subtraction?

$$3 - 2 = 3 + (-2) = 1$$

$$\begin{array}{r} 011 \\ + 101 \\ \hline (1)000 \\ + \quad 1 \\ \hline 001 \end{array}$$

✓

$$3 - 1 = 3 + (-1) = 2$$

$$\begin{array}{r} 011 \\ + 110 \\ \hline (1)001 \\ + \quad 1 \\ \hline 010 \end{array}$$

✓

magnitude range: $-(2^{n-1}-1) \dots (2^{n-1}-1)$

Decimal	1's Complement
3	011
2	010
1	001
+0	000
-0	111
-1	110
-2	101
-3	100

still two zeros

Radix Complement Representation

- 1's complement (or diminished-radix complement) representation can reuse unsigned adder for two operands with opposite signs (theorem)
 - but still two zero representations → need better representation
- A “new” operation: radix complement A^* of an n -digit integer A with radix r defined as

$$A^* = r^n - A$$

- $A^* = (r^n - 1 - A) + 1 \rightarrow$ just the diminished-radix complement + 1
- Example: radix complement of binary integer A
 $\rightarrow$ 2's complement $A^* = 2^n - A$ (simple: just evaluate 1's complement by flipping all bits, then just add 1)

binary number ($n=8$): 01011100 with - sign

2's complement  10100011 + 1 = 10100100



1's complement



2's complement

Radix Complement Representation

- 2's complement representation of n -bit signed (integer) binary numbers
 - Number defined by sign and magnitude $\rightarrow$ start from magnitude A
 - if $A \geq 0$, just take the magnitude (n bits)
 - if $A < 0$, take the 2's complement of the magnitude (n bits)
- Example: can we reuse simple unsigned adder for subtraction?

$$3 - 2 = 3 + (-2) = 1$$

$$\begin{array}{r} 011 \\ + 110 \\ \hline (1)001 \end{array}$$

↑
carry ignored

$$3 - 1 = 3 + (-1) = 2$$

$$\begin{array}{r} 011 \\ + 111 \\ \hline (1)010 \end{array}$$

↑
carry ignored

magnitude range: $-(2^{n-1}) \dots (2^{n-1}-1)$

Decimal	2's Complement
3	011
2	010
1	001
0	000
-1	111
-2	110
-3	101
-4	100

only one zero

Radix Complement Representation

- 2's complement (or radix complement) representation can reuse unsigned adder for two operands with opposite signs (theorem)
 - only one zero representation → definitely preferred representation
 - used in any practical digital implementation
- Common to sign-magnitude and 1's complement representation
 - Positive numbers are the same (MSB behaves like a “sign bit” anyway)
- Summary table (please get familiar with it)

Signed Representations of Binary Numbers

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

Binary-Coded Decimal (BCD)

- BCD is a code to represent ten decimal digits (0 ... 9)

- Each decimal digit represented by a 4-bit number

Decimal → BCD:

Decimal → (5 9 8)₁₀
BCD → 0101 1001 1000



Note:

Six numbers, from **1010 to 1111**,
are not used in BCD.

Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Summary of the Lecture

- Positional number system (radix 10, 2, 8 and 16)
- Conversion among decimal, binary, octal and hex
- Binary arithmetic
- Signed binary number representations
 - SM, 1's complement and 2's complement
- Arithmetic using signed binary numbers
- Binary-coded decimals